

TorchWM

A Modular, Unified Framework for
World Models and Embodied AI

“Build once. Experiment across world models.”

Param Thakkar • github.com/ParamThakkar123/torchwm • MIT licensed • Open source

What TorchWM is

TorchWM is an open-source PyTorch library that brings the major world-model families (recurrent state-space models, transformer/token-based models, diffusion models, and joint-embedding predictive architectures) under one consistent API, together with the environments, datasets, training loops, export paths, and evaluation harnesses they need. Its ambition is to be the layer beneath world-model and embodied-AI research: the place where a *dynamics model*, an *encoder*, an *environment*, and a *policy* are independently replaceable parts rather than assumptions baked into a single research repository.

THE PROBLEM

World-model research is fragmented at the infrastructure layer. Each influential paper ships its own dataset format, environment wrapper, model definition, training loop, config system, and evaluation script. Reproducing a result, comparing two methods fairly, or combining an encoder from one paper with a dynamics model from another is an engineering project before it is a research question.

WHY IT MATTERS

A world model is a learned simulator: it predicts how an environment evolves in response to actions. That single capability underwrites sample-efficient model-based RL, planning in latent space, and embodied agents that can rehearse before acting in the physical world. It is one of the few places where perception, action, and prediction have to be solved jointly, and it is moving fast.

THE LONG-TERM VISION

What PyTorch did for differentiable computation and Gymnasium did for environment interfaces, TorchWM aims to do for world models: a small set of durable abstractions that absorb the infrastructure work, stay out of the way of research, and let a new idea be tested against many models, datasets, and environments without rewriting the scaffolding around it.

Where the project actually stands today

TorchWM is early. It is a working library with a broad implemented surface and a single primary maintainer, not an established piece of ecosystem infrastructure. It has no published benchmark results, no measured adoption to report, and no external contributors beyond incidental patches.

0.4.2

latest PyPI release
(0.5.0 unreleased)

13

registered model
families

11

environment
backends

819

tests collected
in tests/

593

commits since
Nov 2025

52

documentation
pages

How to read this document

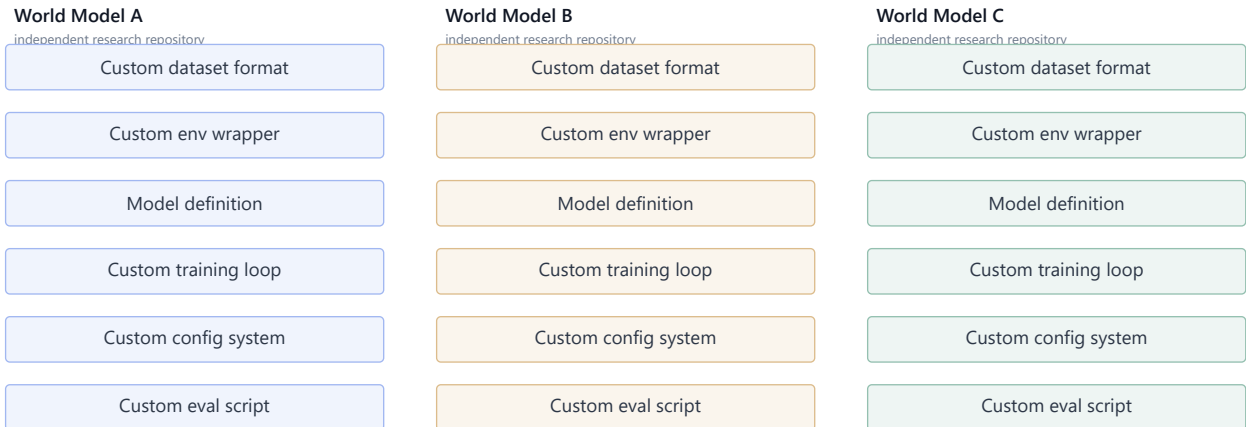
TorchWM is early-stage, and this report is explicit about the boundary between what exists and what is designed. Three markers are used consistently on every page that follows:

IMPLEMENTED working code in the repository today **IN PROGRESS** partially built **PROPOSED** designed, not yet built **VISION** long-term direction

Nothing marked **PROPOSED** or **VISION** should be read as shipped.

Fragmented world-model research

Every serious world-model result of the last few years arrived as a self-contained research codebase. Each one is internally coherent and externally incompatible with the others. The duplication is not in the ideas; it is in everything underneath them.



no shared interface at any layer: six re-implementations of the same six concerns, three times over

Duplicated engineering

Replay buffers, frame stacking, and action normalisation are rewritten per project.

Hard reproduction

Results depend on undocumented wrapper and preprocessing details.

Inconsistent evaluation

Different seeds, episode counts, and aggregation make numbers non-comparable.

Ideas cannot be mixed

Using one paper's encoder with another's dynamics means porting both codebases.

This is documented, not assumed

The fragmentation problem is recognised in the literature. *stable-worldmodel* (Maes et al.) observes that world-model research “remains fragmented and lacks shared benchmarks comparable to those in vision, reinforcement learning, or language modeling,” and points to two contemporary works that independently re-implemented the same Two-Room environment with substantial divergence: a concrete case where the absence of shared infrastructure directly damaged comparability [4]. Recent surveys of world models for embodied AI reach the same conclusion about evaluation [6, 7].

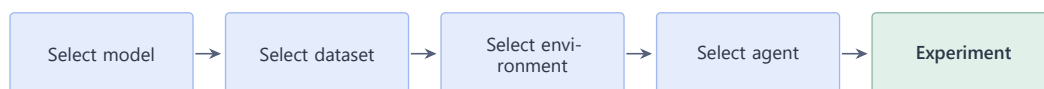
Before TorchWM vs. with TorchWM

BEFORE: the path from idea to first experiment



five infrastructure steps before a single research question is asked

WITH TORCHWM: the same path

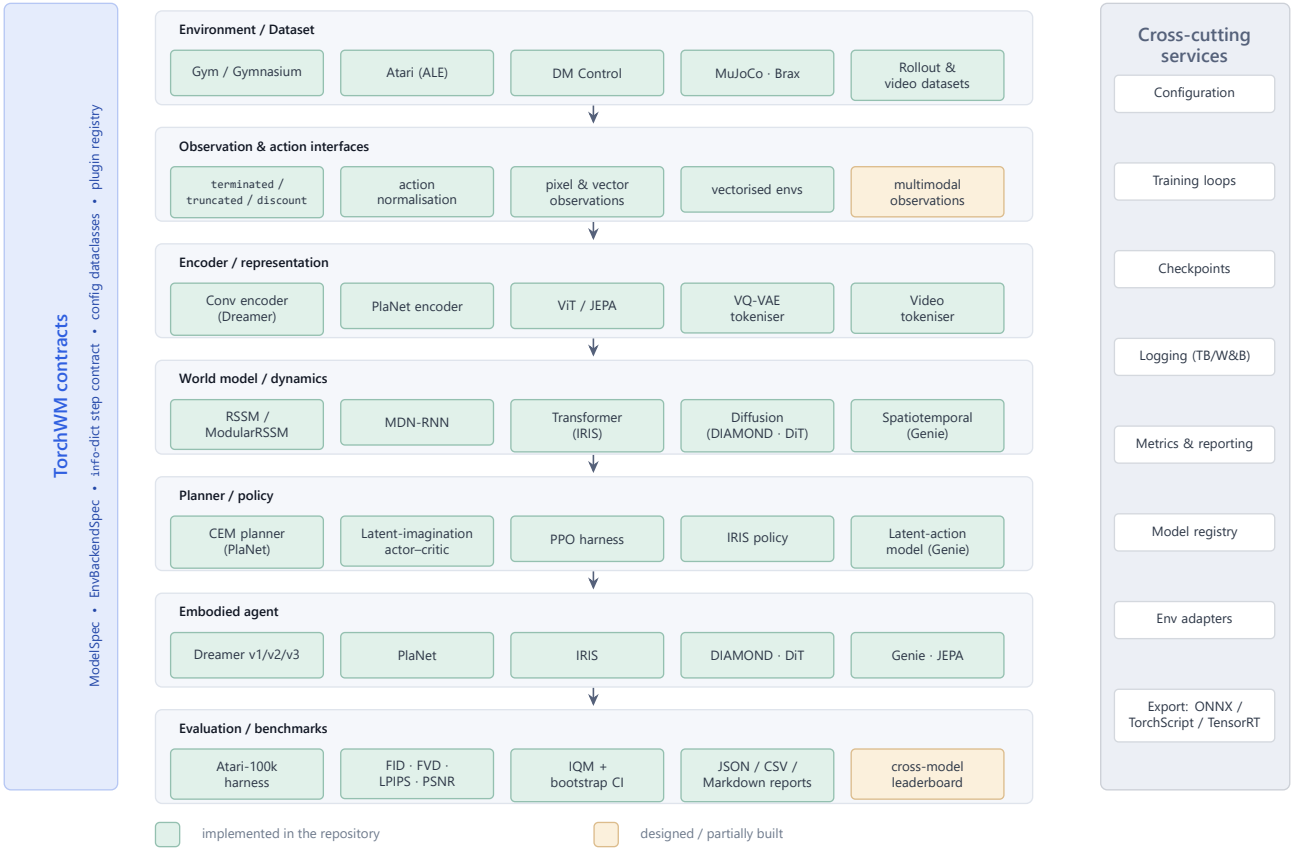


selection replaces re-implementation; the infrastructure is the library's problem, not the researcher's

The claim is specific. TorchWM removes the incidental difficulty (the adapters, wrappers, loops, and config systems) so that the remaining difficulty is the research itself: representation, dynamics, and control. That is the part worth a researcher's time.

Standard contracts, replaceable parts

The architecture is a stack of layers, each defined by a contract rather than an implementation. A layer’s job is to specify what crosses its boundary (observations, latent states, actions, predicted transitions, metrics) so that anything satisfying the contract can be substituted. Boxes below reflect components that exist in the repository today unless marked otherwise.



The contracts that make substitution work

Modularity is only real if it is enforced somewhere. In TorchWM it is enforced by four concrete mechanisms, all of which exist today:

Contract	What it guarantees	Status
ModelSpec	Name, import path, config class, and aliases for every model family, so <code>create_model("iris")</code> works identically to <code>create_model("dreamer-v3")</code> .	IMPLEMENTED
EnvBackendSpec	Name and factory path per backend, so <code>make_env(id, backend=...)</code> is uniform across Gym, ALE, DMC, MuJoCo, Brax, Procggen, DMLab, bsuite, and Unity.	IMPLEMENTED
info-dict step contract	Every adapter and wrapper normalises <code>step()</code> to always return <code>terminated</code> , <code>truncated</code> , and <code>discount</code> , verified by cross-backend contract tests.	IMPLEMENTED
Plugin registry	<code>register_world_model</code> / <code>register_env_backend</code> let external packages appear in <code>list_models()</code> and the factory API without patching TorchWM.	IMPLEMENTED

One design decision worth calling out: the world model is an environment

`WorldModelEnv` wraps a trained dynamics model behind the standard `gym.Env` interface. A latent rollout therefore looks exactly like environment interaction, which means any policy-optimisation code (TorchWM’s own actor-critic and PPO harness, or an external RL library) can train inside an imagined world without knowing it is imagined. This is the single abstraction that connects the model-learning half of the stack to the control half. ■ IMPLEMENTED

What using TorchWM looks like

The API's design goal is that swapping a model family, an environment backend, or a component inside a model should be an edit to a string or a constructor argument, not a fork.

IMPLEMENTED Working today

The following runs against the current package. Construction is unified across all 13 registered model families; the shared step-budget `train()` currently covers the Dreamer family, which is discussed honestly on the next page.

```
import torchwm

# Discover what the registry exposes.
torchwm.list_models()
# ['diamond', 'dit', 'dreamer', 'dreamer-v1',
#  'dreamer-v2', 'dreamer-v3', 'genie', ...]

# One factory, every model family.
agent = torchwm.create_model(
    "dreamer-v3",
    env="walker-walk",      # DeepMind Control
    env_backend="dmc",
    total_steps=1_000_000,
)
agent.train()

# Environments through one backend selector.
env = torchwm.make_env("ALE/Pong-v5",
                      backend="atari")

# Swap the algorithm, keep the code.
for algo in ["dreamer-v1", "dreamer-v2",
            "dreamer-v3"]:
    torchwm.create_model(
        algo, env="Pendulum-v1",
        env_backend="gym", total_steps=20_000
    ).train()
```

IMPLEMENTED Composing below the factory

```
from torchwm import DreamerAgent, ConvEncoder
from torchwm.models import ModularRSSM
from torchwm.envs import WorldModelEnv

# A trained dynamics model, used as a gym.Env:
# Latent rollouts drive ordinary RL code.
imagined = WorldModelEnv(model=world_model)
```

PROPOSED Proposed interface: not yet implemented

The unified `Trainer` / `evaluate` surface below is a design target. The pieces it would sit on (registry, env adapters, benchmark runner, metrics) exist; the single entry point that composes them does not.

```
# PROPOSED API - illustrative, not shipped
from torchwm import (
    make_env, make_dataset, make_encoder,
    make_world_model, make_agent,
    Trainer, evaluate,
)

env = make_env("walker-walk", backend="dmc")
data = make_dataset("rollouts/walker", seq_len=50)

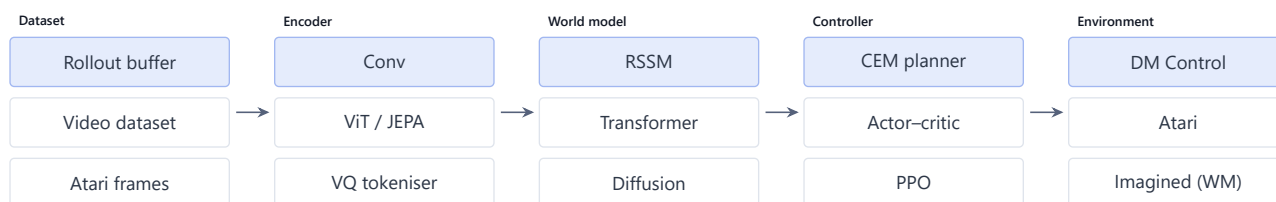
encoder = make_encoder("vq-vae", latent_dim=512)
wm = make_world_model("rssm", encoder=encoder,
                    action_space=env.action_space)
agent = make_agent("actor-critic", world_model=wm)

trainer = Trainer(
    agent, env=env, dataset=data,
    total_steps=1_000_000,
    logger="wandb", checkpoint_every=10_000,
)
trainer.fit()

report = evaluate(
    agent, suite="atari-100k",
    seeds=5, metrics=["iqm", "bootstrap_ci"],
)
```

Every identifier in this block is proposed. It is included to show the intended shape of the abstraction, not to imply availability.

Interchangeability, concretely



Any path through these columns is a valid experiment. The contracts at each boundary are what make the path type-check at runtime rather than at fork-time.

What this buys a researcher

- **Swap world models** without rewriting the data pipeline or the environment stack.
- **Change environments**, from pixels to state or simulator to imagined rollout, behind one selector.
- **Benchmark fairly**, because seeds, episode counts, and aggregation

come from shared code rather than each paper's script.

- **Reproduce** from a serialisable config plus a registry name.
- **Implement something new** by satisfying one contract, then inherit training, logging, checkpointing, export, and evaluation.
- **Publish it** through the plugin registry, so a third-party model is a first-class citizen of `create_model()`.

What exists, what does not

The repository has been under continuous development since November 2025: 593 commits, roughly 37,700 lines of library code across 158 modules, and 819 tests in `tests/` plus a separate benchmark test suite. Seven releases have been published to PyPI (latest 0.4.2; version 0.5.0 sits in the repository unreleased). The work is overwhelmingly by a single maintainer.

IMPLEMENTED

- Unified factory API: `create_config / create_model / make_env`, with 13 registered model families
- Dreamer v1/v2/v3, PlaNet, IRIS, JEPA, DIAMOND, DiT, Genie (3 sizes), ModularRSSM
- 11 environment backends (Gym, ALE, DMC, MuJoCo, Robotics, Procgen, DMLab, Brax, bsuite, Unity, world-model)
- Normalised `step()` info contract with cross-backend contract tests
- `WorldModelEnv`: trained dynamics model exposed as a `gym.Env`
- Encoders/decoders, VQ layer, video tokeniser, RSSM, MDN-RNN, ViT, DDPM/DiT backbones
- Replay/memory modules, dataclass configs with YAML experiment files, serialisation
- Benchmark harness: adapters, multi-seed runner, IQM + bootstrap CI, JSON/CSV/Markdown reports; FID, FVD, LPIPS, PSNR
- Export to ONNX, TorchScript, TensorRT; plugin registry for third-party models and backends
- CLI (`train / eval / benchmark / play`), Docker image, 52 documentation pages
- CI: tests, mypy, lint, CodeQL, docs, PyPI publish; PEP 561 typing stubs

IN PROGRESS

- **Uniform training entry point.** Construction is unified across all 13 model families, but a shared step-budget `train()` currently covers the Dreamer family and a subset of others. Each remaining family still has its own trainer module.
- **Uniform evaluation entry point.** `evaluate()` exists on some agents; the benchmark adapters paper over the rest.
- **Paper-alignment verification.** Test suites pinning implementations to their papers' appendices exist for IRIS, DiT, and Genie; other families are not yet covered.
- **Dataset abstraction.** Rollout, sequence, and video datasets exist, but there is no single dataset contract every trainer consumes.
- **Multimodal observations** beyond pixels + proprioceptive vectors.
- **Minecraft / open-world environment** adapter.
- **Reference results.** The benchmark harness runs; no reproduced published numbers have been produced or validated yet.

PLANNED

- Single Trainer / evaluate surface spanning every registered family (the proposed API on page 4)
- Published, seed-controlled reference results with confidence intervals for each implemented family
- Standard dataset contract plus conversion tools for common trajectory formats
- Cross-model, cross-environment leaderboard generated by the benchmark harness
- Distributed and multi-GPU training paths
- Documented model/dataset hub so third-party components are installable and discoverable
- Robotics-oriented environments and sim-to-real evaluation protocol
- Community contribution process for models, adapters, and benchmark tasks

Early researcher validation

The problem framing and architecture described in this report have been discussed directly with researchers and engineers working on world models and embodied AI at organisations including **Google DeepMind**, **Odyssey**, **General Intuition**, and **Kog AI**. The reactions to the project were encouraging, and the conversations supported the premise that infrastructure fragmentation and duplicated engineering are real, felt costs in this area of research.

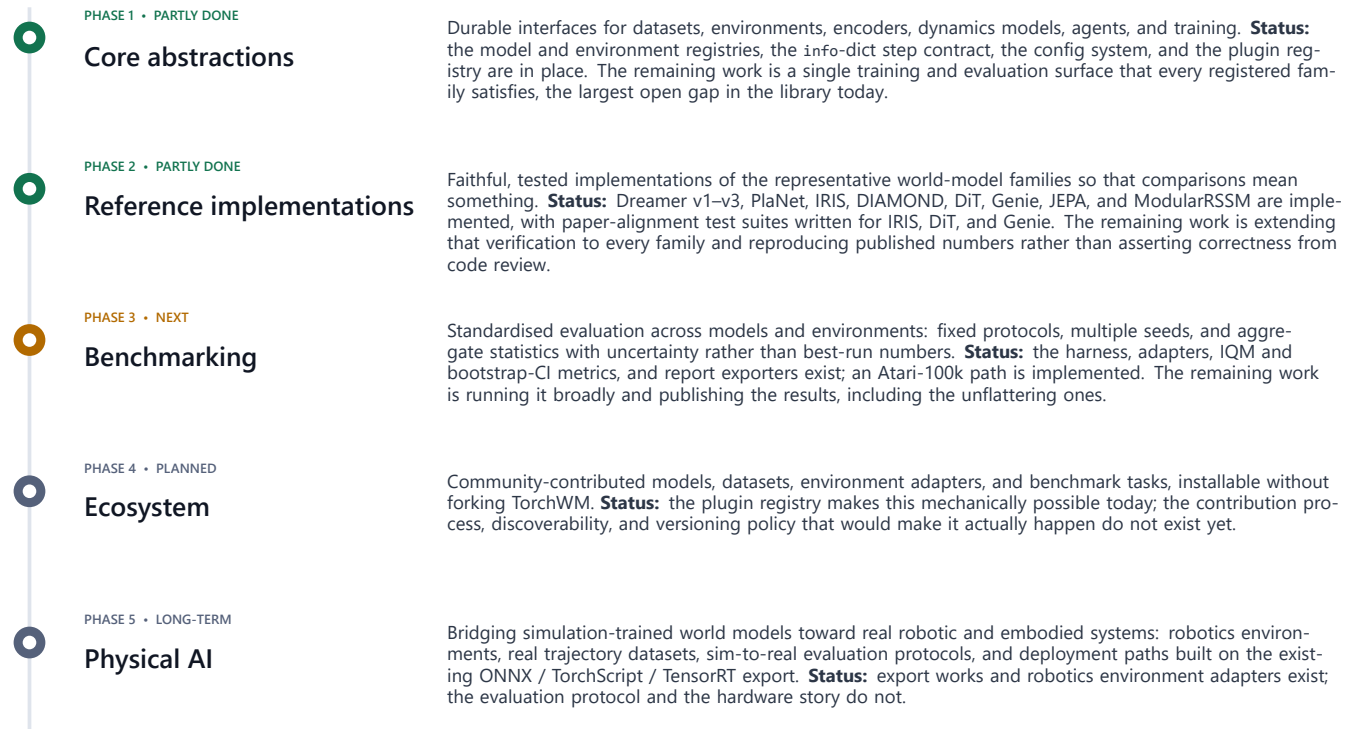
What these conversations were, precisely: informal technical discussions with individual researchers. They are reported here as *feedback and problem validation only*. They are not partnerships, endorsements, sponsorships, customer relationships, or any form of official organisational support. None of the organisations named has reviewed, approved, or endorsed TorchWM, and no individual is quoted in this document.

How the feedback shaped the project: it reinforced three decisions that are visible in the architecture: (i) treating the environment interface as a hard contract with normalised termination and discount semantics, because wrapper divergence is a recurring source of irreproducibility; (ii) making the trained world model usable as an ordinary environment, so that model learning and policy learning stay decoupled; and (iii) prioritising a benchmark harness with proper aggregate statistics (IQM, bootstrap confidence intervals) over headline single-seed numbers.

Stated plainly: TorchWM has no measured adoption, no published benchmark results, no external funding, and one primary maintainer. The case for it rests on the problem being real and the design being sound, not on traction that does not yet exist.

From library to infrastructure

The phases below are sequenced by dependency, not by date. Each one is only worth starting when the previous one is solid enough to build on; publishing benchmark numbers before the training paths are uniform, for example, would produce exactly the incomparable results the project exists to fix.



The larger vision

World models sit at the intersection of four communities that currently share very little code: representation learning, model-based reinforcement learning, simulation, and robotics. Each has its own conventions for what an observation is, what an action is, what a rollout is, and what counts as a result. A world model needs all four at once, which is why it is the natural place for a shared infrastructure layer to form.

TorchWM's long-term goal is to be that layer: a common substrate where a dynamics model trained on video can be evaluated as a simulator, used as an environment for policy learning, benchmarked against alternatives under a fixed protocol, and exported toward a physical system, without any of those steps requiring a rewrite. VISION

Open questions the design still has to answer

What is a latent state, generically?

An RSSM carries a deterministic recurrent state plus a stochastic variable; IRIS carries discrete tokens; a diffusion world model carries pixels and a noise level; JEPa carries a continuous embedding. A contract loose enough for all four risks being an opaque blob. TorchWM's current answer (model-owned state, with the contract enforced only at the environment boundary) is a compromise, not a settled result.

How should imagined rollouts report their own unreliability?

`WorldModelEnv` makes latent rollouts look like ordinary environment steps, which is what makes it useful and also what hides compounding prediction error from the policy code above it. A fixed imagination horizon is blunt; surfacing per-step model uncertainty through `info` is more honest but has no cross-family definition yet.

Can action spaces be unified across discrete, continuous, and latent actions?

Genie infers latent actions from unlabelled video; Dreamer and IRIS consume environment actions directly; robotics adds continuous control at a different frequency from perception. One action interface spanning all three is desirable for composition and not obviously well-posed.

Where TorchWM fits, and where it does not

Adjacent efforts exist and are worth taking seriously. TorchWM's case does not rest on being alone in the space; it rests on covering a combination the existing tools do not.

Effort	Focus	Relationship to TorchWM
Gymnasium [1]	Standard environment interface for RL.	A dependency and a model. TorchWM adopts its step semantics and extends them with a normalised info contract across nine backends.
MBRL-Lib [2]	Modular model-based RL toolbox: dynamics models, planners, MBRL algorithms.	Closest in spirit for the control half. TorchWM spans a wider model space: token-based, diffusion, and video world models, not only proprioceptive MBRL.
TorchRL [3]	General PyTorch RL library with tensor-first data structures.	Complementary. TorchWM's <code>WorldModelEnv</code> lets an imagined environment be consumed by external RL libraries rather than replacing them.
stable-worldmodel [4]	Reproducible world-modelling platform: Lance data layer, planning solvers, controlled factors of variation.	Nearest neighbour. Strongest on data infrastructure and MPC-style planning evaluation; TorchWM is broader on model families and the agent/policy side, and PyTorch-native throughout.
OpenWorldLib [5]	Unified codebase and definition spanning generative, reasoning, and interactive world models.	Broader in the generative/multimodal direction; less focused on the reinforcement-learning and control loop that TorchWM centres.
WorldArena [8] and related benchmarks	Evaluation suites for embodied world models.	Evaluation targets rather than infrastructure. TorchWM's harness is meant to run against suites like these, not compete with them.

Where TorchWM could differentiate

- **Breadth of model families under one registry.** Recurrent state-space, transformer/token, diffusion, and joint-embedding predictive models are reachable through the same factory, a span most alternatives do not attempt.
- **The world model as an environment.** Making imagination a `gym.Env` keeps model learning and policy learning independently replaceable, and lets external RL code operate on learned dynamics unmodified.
- **PyTorch-native and deployment-aware.** One framework end to end, with ONNX / TorchScript / TensorRT export already in place, an unusual concern for a research library, and the prerequisite for the Physical AI phase.
- **Contracts over conventions.** Termination, truncation, and discount semantics are enforced by tests across every backend rather than documented and hoped for.

None of these is a claim of superiority. They are the axes along which TorchWM is being built, stated so that they can be judged.

References

- [1] M. Towers et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. arXiv:2407.17032.
- [2] L. Pineda, B. Amos, A. Zhang, N. O. Lambert, R. Calandra. *MBRL-Lib: A Modular Library for Model-Based Reinforcement Learning*. arXiv:2104.10159.
- [3] A. Bou et al. *TorchRL: A data-driven decision-making library for PyTorch*. arXiv:2306.00577.
- [4] L. Maes, Q. Le Lidec, D. Haramati, N. Massaudi, D. Scieur, Y. LeCun, R. Balestrieri. *stable-worldmodel: A Platform for Reproducible World Modeling Research and Evaluation*. arXiv:2602.08968 / arXiv:2605.21800.
- [5] OpenDCAI. *OpenWorldLib: A Unified Codebase and Definition of Advanced World Models*. arXiv:2604.04707.
- [6] A *Comprehensive Survey on World Models for Embodied AI*. arXiv:2510.16732.
- [7] *World Models: A Comprehensive Survey of Architectures, Methodologies, Reasoning Paradigms, and Applications*. arXiv:2606.00133.
- [8] *WorldArena: A Unified Benchmark for Evaluating Perception and Functional Utility of Embodied World Models*. arXiv:2602.08971.
- [9] D. Ha, J. Schmidhuber. *World Models*. NeurIPS 2018. arXiv:1803.10122.
- [10] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, J. Davidson. *Learning Latent Dynamics for Planning from Pixels (PlaNet)*. ICML 2019.
- [11] D. Hafner, T. Lillicrap, J. Ba, M. Norouzi. *Dream to Control: Learning Behaviors by Latent Imagination (Dreamer)*. ICLR 2020.
- [12] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. *Mastering Diverse Domains through World Models (DreamerV3)*. arXiv:2301.04104.
- [13] V. Micheli, E. Alonso, F. Fleuret. *Transformers are Sample-Efficient World Models (IRIS)*. ICLR 2023.
- [14] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, F. Fleuret. *Diffusion for World Modeling: Visual Details Matter in Atari (DIAMOND)*. NeurIPS 2024.
- [15] J. Bruce et al. *Genie: Generative Interactive Environments*. ICML 2024.
- [16] M. Assran et al. *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)*. CVPR 2023.
- [17] W. Peebles, S. Xie. *Scalable Diffusion Models with Transformers (DiT)*. ICCV 2023.
- [18] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, M. G. Bellemare. *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. NeurIPS 2021. (Source of the IQM and bootstrap-CI protocol used in TorchWM's benchmark harness.)